

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice

Week 1: Fundamentals of Deep Learning for Generative AI

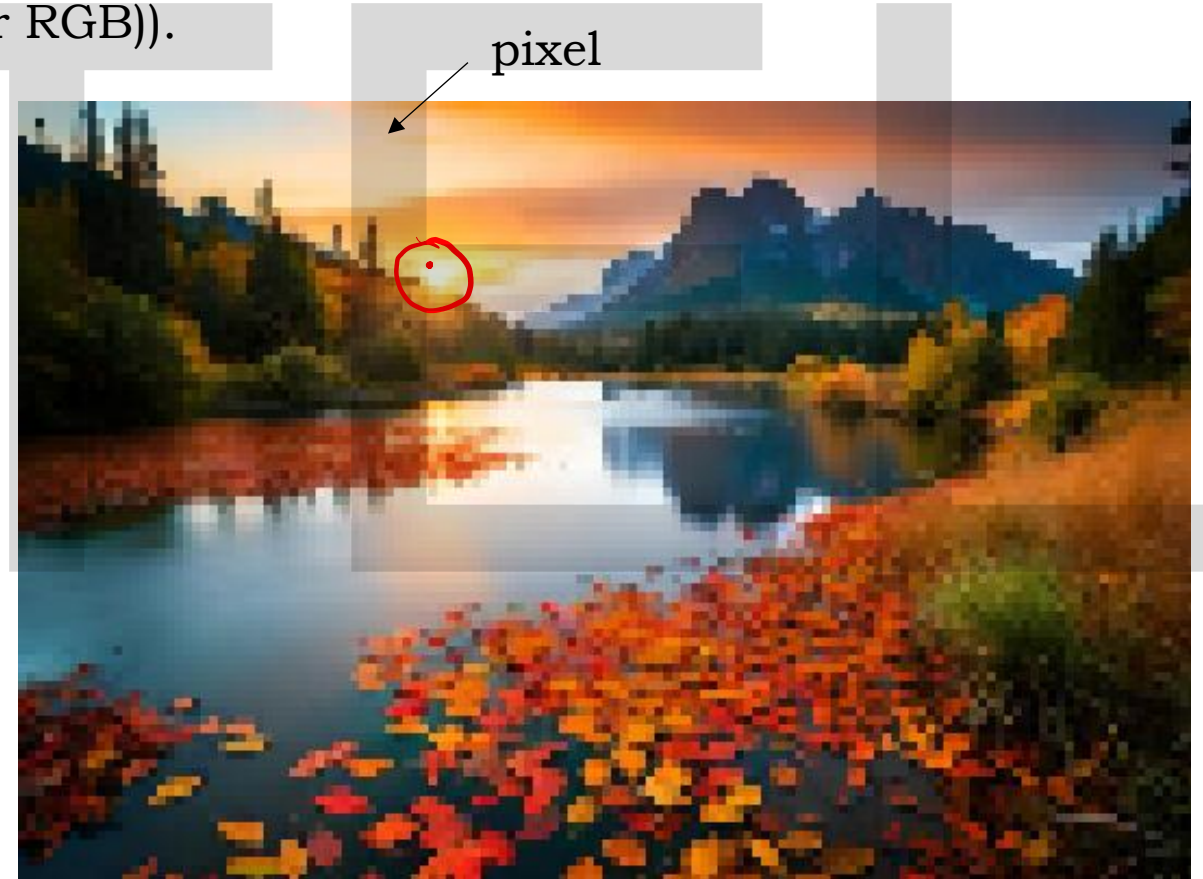
Today's Session Overview

- How does a computer interpret an image?
- Why are fully connected networks not ideal for image data?
- Introduction to Convolutional Neural Networks (CNN)
 - Key components of a CNN
 - Understanding the Convolution Layer
 - Padding
 - Stride
 - Filters

How does the system read an image?

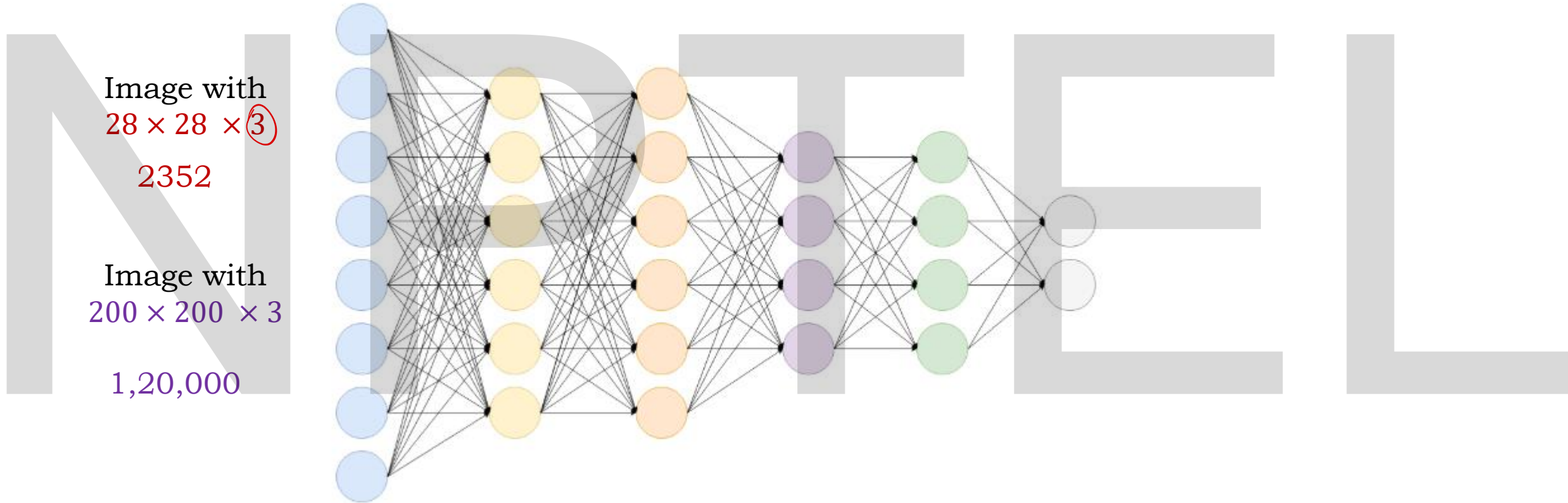
Each pixel in an image is represented by a numerical value. These numerical values are called **pixel values**.

An image can be represented as a **matrix of pixel values**; these pixel values denote the intensity of the pixels (brightness (for grayscale) or color intensity (for RGB)).



The **dimensions of an image** are defined by the number of pixels along its height and width.

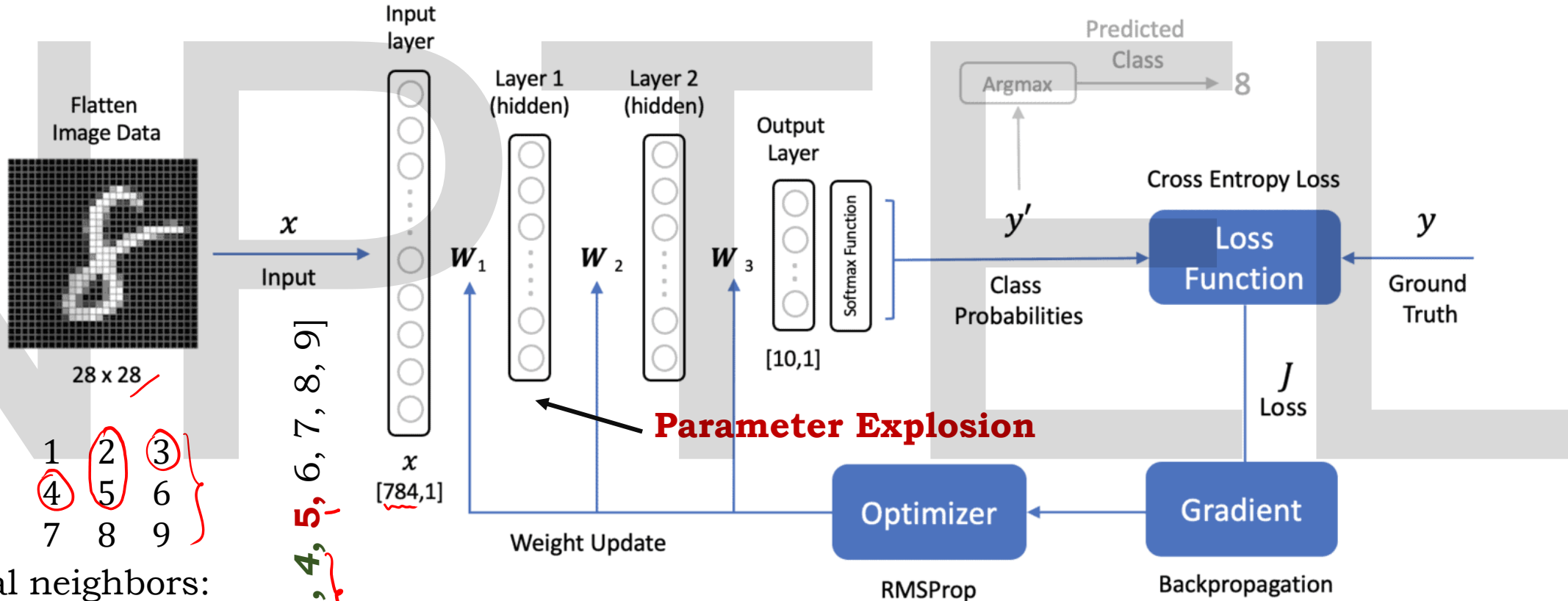
Why not fully connected networks? (MLP)



Example

Loss of Spatial Structure:

- Flattening images into 1D input vectors **destroys spatial relationship** (how pixels are positioned and related to each other in space).
- Fully connected NN treats every pixel as an independent feature.
- Local dependencies (edges, corners, textures) are lost.



Horizontal neighbors:
1-2-3, 4-5-6, 7-8-9

Vertical neighbors:
1-4-7, 2-5-8, 3-6-9

Horizontal neighbors:
1-2-3, 4-5-6, 7-8-9

Vertical neighbors:
1-4-7, 2-5-8, 3-6-9

18-06-2026

[1, 2, 3, 4, 5, 6, 7, 8, 9]

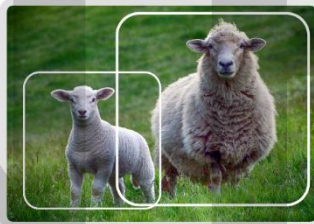
- ✓ Slow Training and
- ✓ Increases the risk of overfitting

Introduction to CNN

- A convolutional neural network (CNN or ConvNet) is a specialized **deep learning model designed for processing data such as images.**
- It has been widely used in **computer vision tasks** like **image classification, object detection, and image segmentation.**



Image Classification



Object Detection

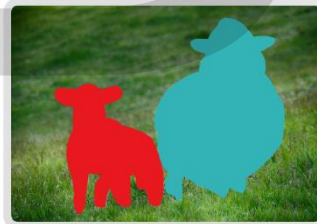
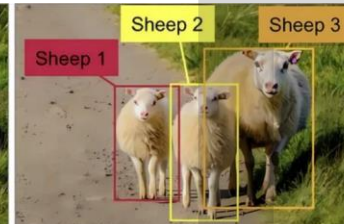


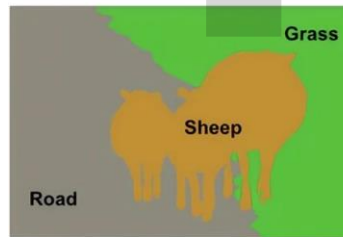
Image Segmentation



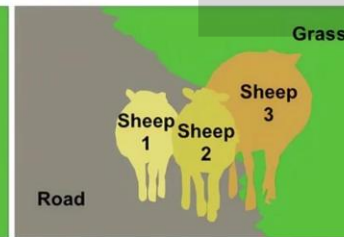
Classification + Localization



Object Detection



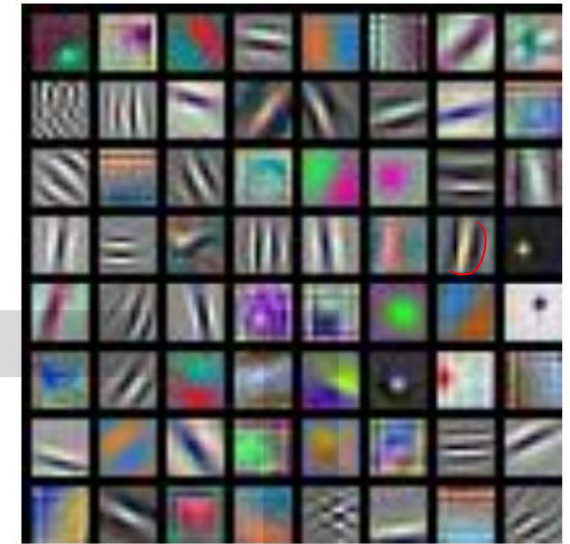
Semantic Segmentation



Instance Segmentation

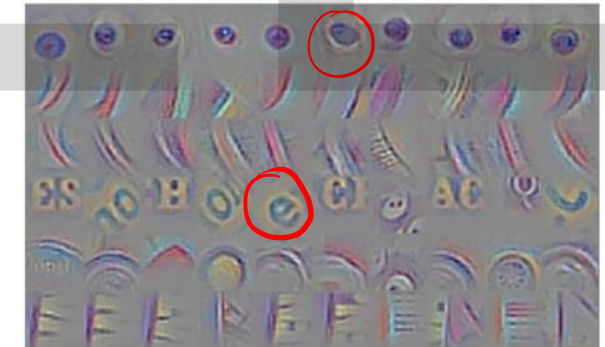
- CNNs mimic the human visual system — they focus on **local regions** **first (lines, edges, corners)** and gradually understand **the whole image** **through multiple layers.**

First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)



AlexNet: 64 filters, each 3x11x11

Deeper conv layers: Harder to visualize
Tend to learn larger structures e.g. eyes, letters



6th layer conv layer from an ImageNet model

Visualization from [Springenberg et al, ICLR 2015]

Key Components of a Convolutional Neural Network (CNN)

Input

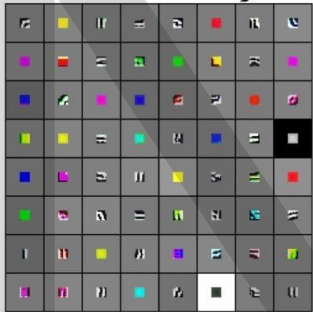


Low-level features

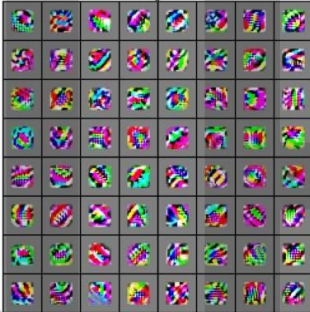
Mid-level features

High-level features

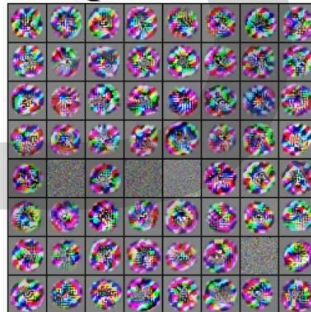
Linearly separable classifier



VGG-16 Conv1_1



VGG-16 Conv3_2

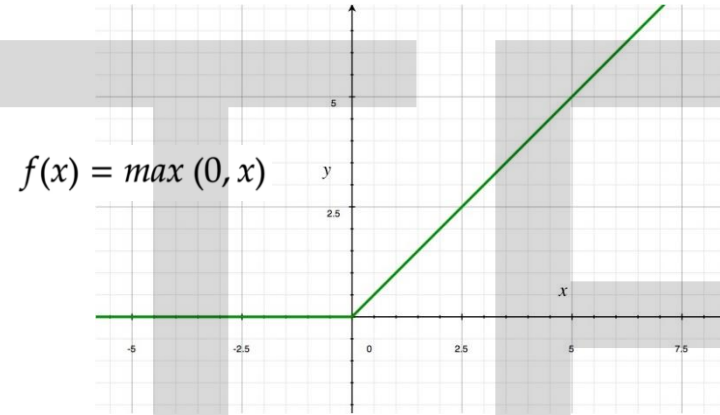


VGG-16 Conv5_3

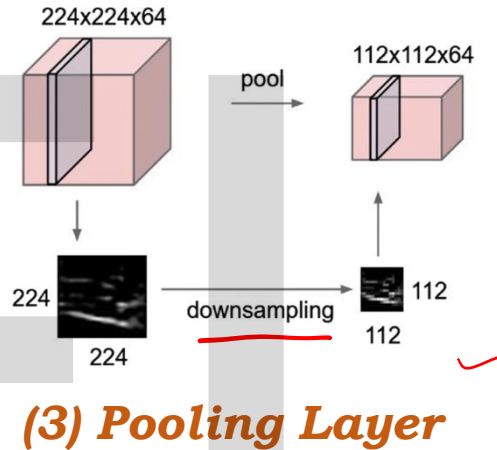
(1) Convolutional Layer

5. Normalization Layer

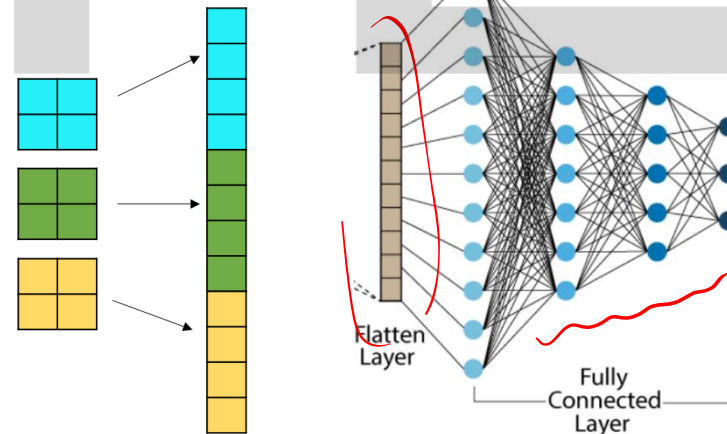
6. Dropout (Optional)



(2) Activation Function (ReLU)



(3) Pooling Layer



(4) Fully Connected Layer

Classification



Convolutional Layer

A **convolutional layer** is the fundamental building block of a CNN that extracts features from input image by applying a set of learnable filters (kernels). It works by sliding these filters over the input data, performing a convolution operation (element-wise multiplication and summation) to create a feature map.

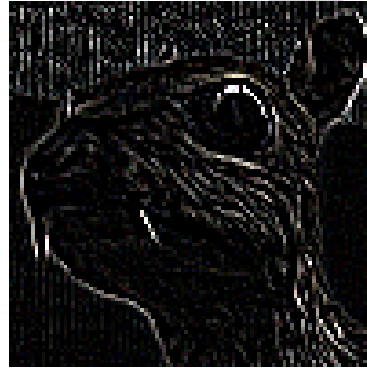
Input image



Convolution Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



1	0	1	0	1	0
0	1	1	0	1	1
1	0	1	0	1	0
1	0	1	1	1	0
0	1	1	0	1	1
1	0	1	0	1	0

Input

1	0	1
0	1	1
1	0	1

1	2	3
4	5	6
7	8	9

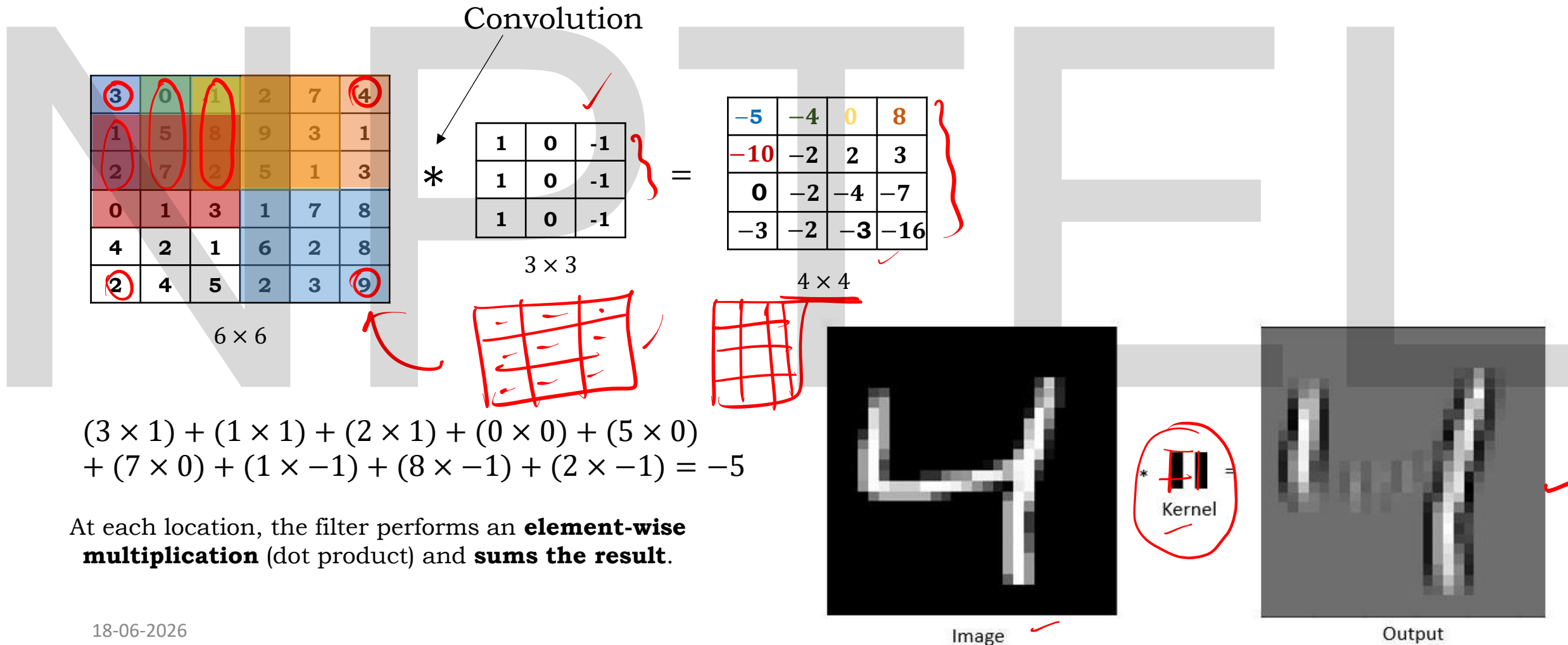
filter (kernel)

31			

Feature map

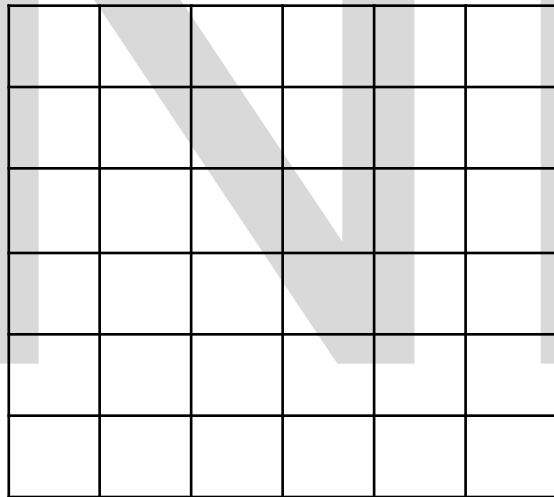
Feature hierarchy: **Early CNN layers** detect **simple features** (edges, corners),
While **deeper layers** capture **complex patterns** (object parts and structures).

CNN: Convolutional operation



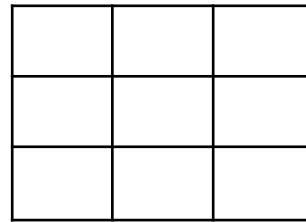
Mathematics of 2D convolution in a CNN

How the **feature map size** is calculated when applying a **convolutional filter (kernel)** to an input image with a stride of 1.



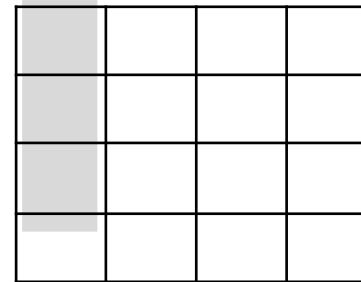
6×6 ($n \times n$)

*



3×3 ($f \times f$)

=



Feature map

???

4×4

Problem: Feature maps shrink with each layer!

Output in general:

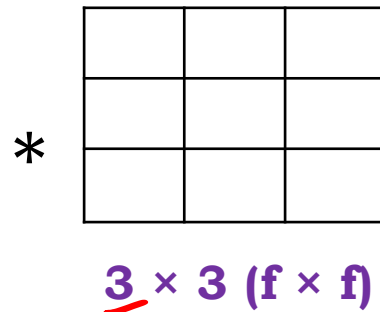
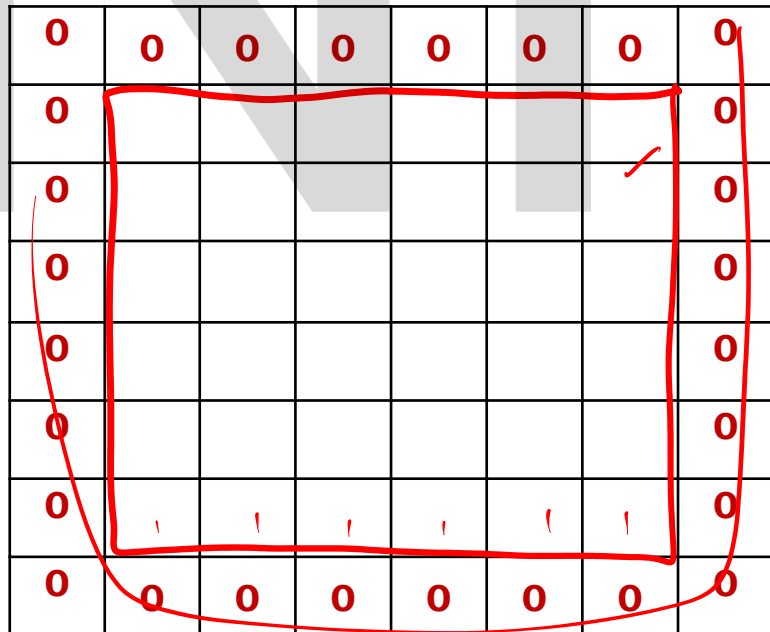
$$\begin{aligned} & (n - f + 1) \times (n - f + 1) \\ &= (6 - 3 + 1) \times (6 - 3 + 1) \\ &= 4 \times 4 \end{aligned}$$

Solution: Add padding around the input before sliding the filter

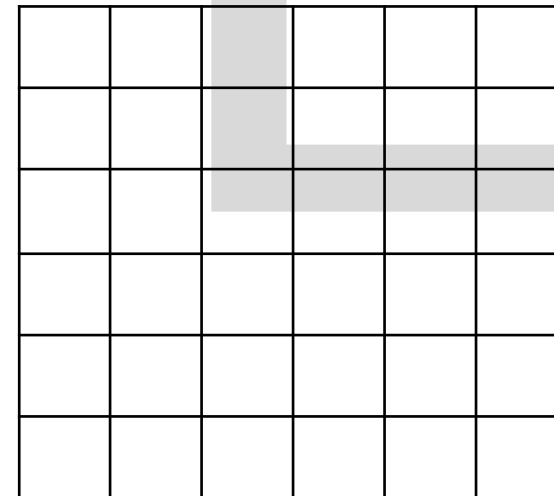
Padding in Convolutional NN

Why Padding is required ?

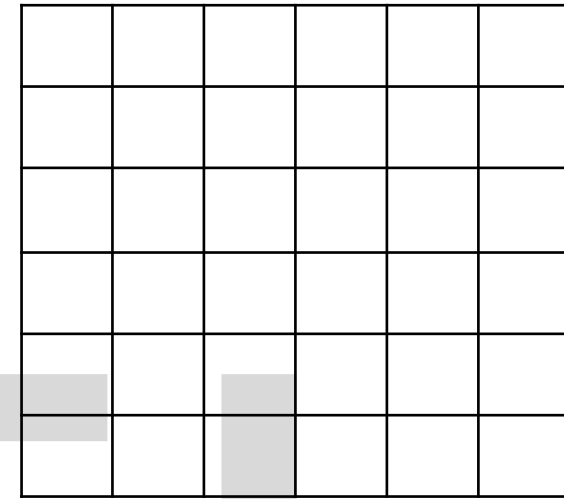
- During convolution, the size of the output feature map is determined by
- If we simply apply the kernel on the input, then the output feature map will be **smaller than the input**.
- This reduction in size leads to loss of information at the borders of the input image. In order to preserve the border information and maintain the spatial dimension, **we use padding, padding is applied around the input**.



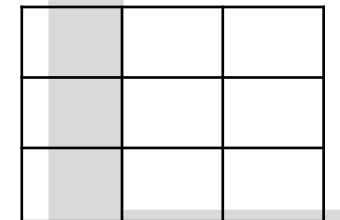
=



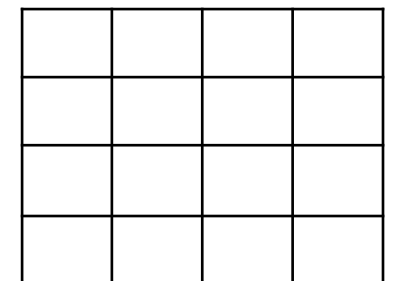
$$(n + 2p - f + 1) \times (n + 2p - f + 1)$$



6×6 ($n \times n$) Input



3×3 ($f \times f$) Kernel



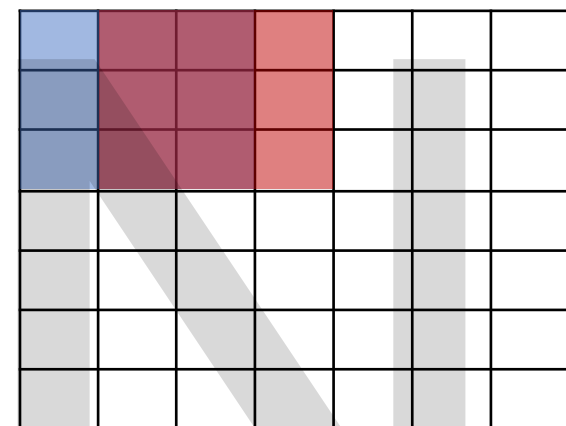
Output feature map
 4×4

$$(n - f + 1)$$

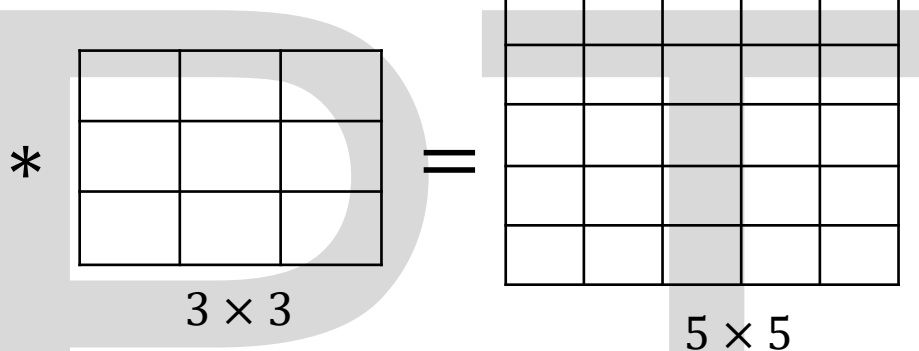
$$(n + 2p - f + 1) = 6 + 2 - 3 + 1 = 6$$

Stride

In Keras Conv2D, the strides parameter controls how the filter moves over the input.



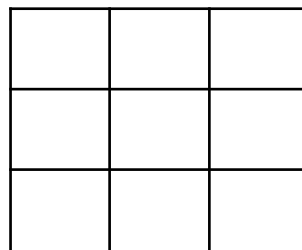
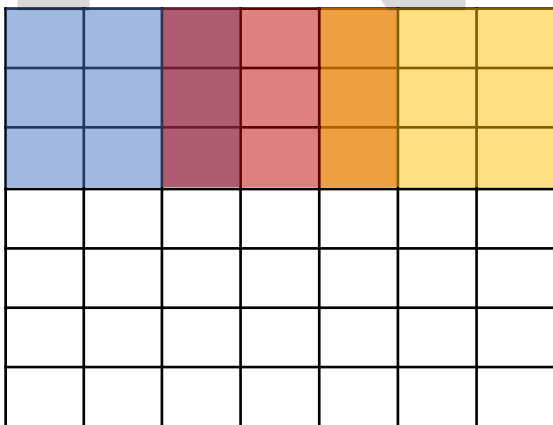
7 × 7



3 × 3

5 × 5

$$\frac{7 + 0 - 3}{1} + 1 \times \frac{7 + 0 - 3}{1} + 1 = 5 \times 5$$



Stride = 2

Output = 3 × 3

$n \times n$ image

$f \times f$ kernel/filter

p = padding

s = stride

Then output size is calculated as

$$\frac{n+2p-f}{s} + 1 \times \frac{n+2p-f}{s} + 1$$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$\lfloor 3.5 \rfloor = 3$$

$$\lfloor 3.1 \rfloor = 3$$

$$\lfloor 3.9 \rfloor = 3$$

The floor function always gives the **greatest integer less than or equal to** the number.

So, both 3.1 and 3.9 “fall down” to **3**

Stride and Padding Implementation in Keras

```
from tensorflow.keras.layers import Conv2D
```

```
# SAME padding with stride=2
```

```
Conv2D(32, (3,3), strides=(2,2), padding='same', input_shape=(64,64,1), activation='relu')
```

```
# VALID padding with stride=2
```

```
Conv2D(32, (3,3), strides=(2,2), padding='valid',  
input_shape=(64,64,1), activation='relu')
```

```
# No padding (valid)
```

```
Conv2D(32, (3,3), padding='valid')
```

```
# Same padding (output height/width same as input)
```

```
Conv2D(32, (3,3), padding='same') ✓
```

When the input image has different aspect ratio (e.g., 128 × 256), then the output feature map is rectangular, not square. In such case, stride values differ along height and width, such as

strides = (2, 3)

move 2 pixels down ✓

move 3 pixels right ✓

Stride = 1 which is by default

Next session

✓ ReLU, Pooling Layer, Flatten Layer in CNN

NPTEL

NIPTEEL

Thank you